Programming Assignment 2

DA6400: Jan-May 2026

1 Team Details

Kevin Kinsey S	Madhav Tadepalli	Soumya Lodha
EP23B027	EE23B040	EE23B140

1.1 Work Split

The work split was even across all tasks, with each team member running different hyperparameter configurations till we found the optimal configuration. After that, we ran all other questions together parallelly to increase time efficiency.

1.2 Links

Drive: <>

Video: https://drive.google.com/file/d/1Yz7aYQQbFo0fHScnU9IvOemxA9LcbbeB/view?usp=sharing

GitHub: https://github.com/Maddy-1111/Reinforcement-Learning/tree/main

3. Deep Q-Networks on Mountain Car-v0

Q1 Vanilla DQN Algorithm

The implementation follows the standard DQN architecture as specified, utilizing an Adam optimizer and a dual-hidden-layer MLP. The code for simple vanilla DQN algorithm is attached in the links. We used two hidden layers with 64 neurons each for the DQN. The following table summarizes the optimized hyperparameter set used for these runs:

Hyperparameter	Value
Learning Rate	5×10^{-4}
Gamma (γ)	0.99
Batch Size	128
Buffer Size	20,000
ε Schedule	1.0 *to* 0.05
ε Decay Steps	100,000
Target Update (I)	2,000
Max Steps / Episodes	2,000 / 1,000

All the runs utilised for inference, has been consolidated into `results.jsonl` file in the shared drive, which contains the returns for all seeds and hyperparameter configurations.

Q2 Random Seed Runs of Standard DQN

We ran the standard DQN algorithm with the above hyperparameters for 15 different random seeds. The return curves for all seeds are shown in the Figure 1.

The return curves show that the agent gradually improves from consistently poor returns to near-optimal performance, indicating that DQN successfully learns a stable strategy for reaching the goal. During early training, the car fails to reach the hill and oscillates near the valley, resulting in long episodes with low returns. As learning progresses, the agent discovers a more efficient trajectory and the variance across seeds decreases, showing convergence to a consistent policy.

The optimal behaviour learned by the car is not to directly accelerate the target. As seen from the video (`visualisation.mp4`):

1. The car first moves away from the goal (to the left), climbs the opposite hill, and then accelerates forward to build sufficient momentum to reach the right hill.
2. If the accumulated momentum is still insufficient, the car oscillates back and forth (moving slightly forward, then backward again) until it gains enough velocity to reach the top

This behaviour emerges because the engine power alone is insufficient to climb the hill directly; the agent must exploit gravitational potential energy by swinging to build speed.

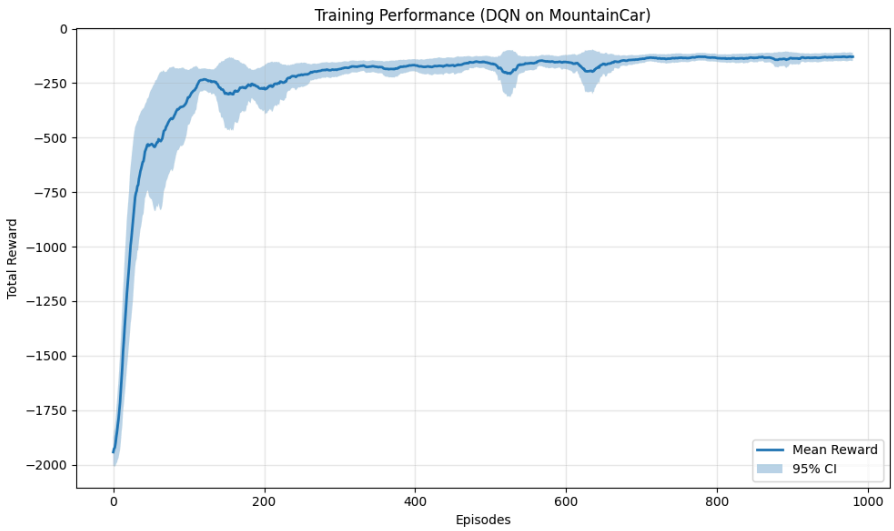

Figure 1: 2000 steps

Q3 Length of Episodes

a) Plots

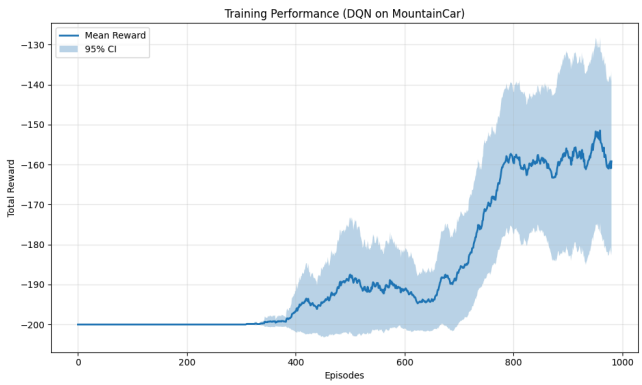

Figure 2: 200 steps

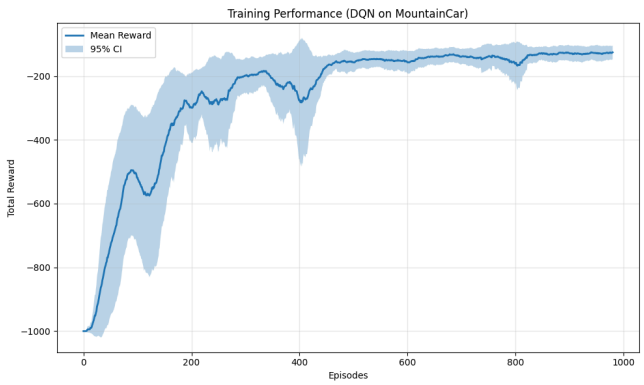

Figure 3: 1000 steps

b) Comparison from the plots

With a truncation length of **200 steps**, the agent struggles to learn an effective policy.

- There is a long flat region indicating that the agent is not making meaningful progress. The learning starts quite late showing that the agent is not able to explore effectively.
- The variance across seeds is high, suggesting that the learning process is unstable and heavily dependent on the initial conditions.

With a truncation length of **1000 steps**, the agent shows significant improvement in learning:
- The agent starts to learn a better policy much earlier, around episode 100, and the returns improve more rapidly compared to the 200-step case, indicating that the agent is able to explore more effectively and discover better strategies.
- Policy convergence (Stability) is not fully achieved by episode 1000, as the returns are still improving and show significant variability.

With a truncation length of **2000 steps**, the agent learns the optimal policy much more effectively:
- The agent starts learning a good policy very early, around episode 50, and the returns improve rapidly, reaching near-optimal performance by episode 200.
- The variance across seeds decreases significantly as training progresses, indicating that the learning process is more stable and less dependent on initial conditions.

Overall, the results clearly demonstrate that a **longer truncation length** allows the agent to explore more of the state space and **learn a more effective policy**, while a shorter truncation length severely limits learning and leads to poor performance.

c) Introduction of Truncation Length

Though the original Mountain Car environment does not have a truncation length, we introduced it to limit the maximum number of steps per episode. This was done to:

1. Prevent excessively long episodes that can occur when the agent fails to reach the goal, which can lead to inefficient learning and increased training time.
2. Allow consistent comparison across different runs and hyperparameter configurations.
3. Avoid the replay buffer being filled with very long episodes that do not contribute to learning, which can degrade the quality of experience replay and slow down convergence.

d) Harm of Short Truncation Length

A short truncation length (e.g., 200 steps) can harm learning in the Mountain Car environment because it does not allow the agent enough time to explore the state space and discover effective strategies for reaching the goal.

1. The optimal policy for Mountain Car involves building momentum by swinging back and forth,which can take many steps to achieve. If the episode is truncated too early, the agent may never have the opportunity to learn this strategy, leading to consistently poor returns and slow learning progress.
2. The agent may become stuck in a local minimum, where it fails to make meaningful progress and receives low rewards, further hindering learning and leading to high variance across different runs.

Overall, a short truncation length can severely limit the agent's ability to learn an effective policy and achieve good performance in the Mountain Car environment.

e) Optimal Truncation Length

As seen from the plots,
- **Truncation lengths of 200:** Leads to delayed learning and poor performance due to insufficient exploration.

- **Truncation lengths of 1000:** Shows improvement against 200, has higher variance than 2000 but does fully converge by episode 1000.
- **Truncation lengths of 2000:** Enables early learning, rapid improvement, and stable convergence to a near-optimal policy.

Hence, we can infer that the optimal truncation length for this environment can be between 1000 and 2000 steps, as from 1000 steps onwards the agent starts to learn effectively, and at around 2000 steps that we see consistent convergence across seeds.

Q4 DQN with Replay Factor

a) Sweep across different replay factors: $\rho = \{1, 2, 4, 8\}$

i) Comparison plots

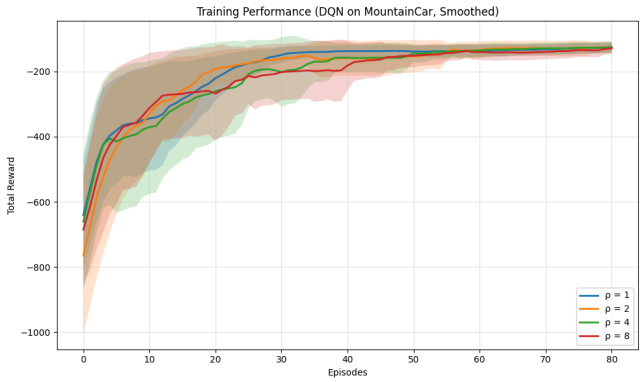

Figure 4: Reward over episodes(in multiples of 10) with replay factor (over multiple seeds)

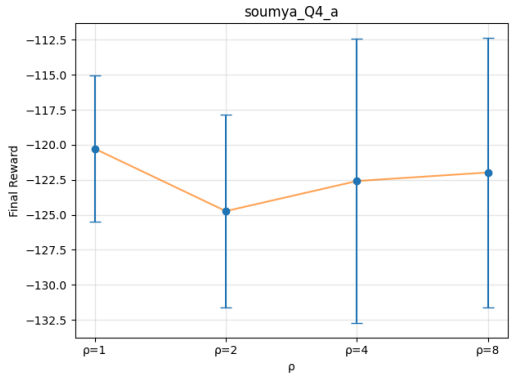

Figure 5: Final return distribution across seeds with replay factor

ii) Analysis of different replay factors

- **$\rho = 1$:** Shows the fastest initial learning, with returns improving rapidly in the first 200 episodes and happens to have the best final performance.
- **$\rho = 2$:** Also learns quickly but slightly slower than $\rho = 1$, but it's marked with quite some variability across seeds, especially in the early episodes, but eventually converges to a similar performance as $\rho = 1$.
- **$\rho = 4$:** Learns more slowly but eventually converges to a similar final performance.
- **$\rho = 8$:** Shows the slowest learning, with a very gradual improvement in returns. This happens to be the most unstable variant, with high variability across seeds, especially in the early episodes. However, it still converges to a similar final performance as the other variants.

The final performance comparison plot shows that $\rho = 1$ achieves the best mean return, followed by $\rho = 4$, while $\rho = 2$ and $\rho = 8$ perform slightly worse. However, the 95% confidence intervals overlap substantially across all ρ values, indicating that the differences in final performance are not statistically significant.

iii) Effect of Increasing of Replay Factor:

Increasing the replay factor means performing more gradient updates per environment step using the same replay buffer. This has two competing effects:

Positive Effects:

- Improves data reuse, allowing the agent to learn more effectively from each experience.

Negative Effects:

- Updates become more correlated, which can lead to instability in learning and divergence if the replay factor is too high.

- Reduces sample complexity, as the agent can learn more from fewer interactions with the environment.
- Can extract more information from each transition, potentially leading to faster learning and better performance.
- Risk of everfitting to stale replay data, especially if the replay buffer is not large enough to provide diverse experiences.
- Slower adapatation and learning to new policy distributions.

Given that this problem is quite a simple one, all of the replay factors are able to eventually learn a near-optimal policy, and the main effect of increasing ρ is to slow down learning and increase variability rather than improving final performance.

iv)

When Increasing Replay Factor is Beneficial:

- When environment interactions are expensive or limited, increasing the replay factor can improve sample efficiency by learning more from each experience.
- When the replay buffer is sufficiently large and diverse, a higher replay factor can help the agent learn more effectively without overfitting to stale data.
- When computational resources are not a constraint, increasing the replay factor can allow for more thorough learning from each experience, potentially leading to better performance.

Example: Robotics Training on Real Hardware: Collecting a single trajectory requires physical movement and time. Using larger ρ allows the agent to reuse collected data more extensively and learn with fewer real-world interactions.

When Increasing Replay Factor is Harmful:

- When the replay buffer is small or lacks diversity, a high replay factor can lead to overfitting to stale data and instability in learning.
- When computational resources are limited, increasing the replay factor can lead to significantly longer training times without a corresponding improvement in final performance.
- When the environment is non-stationary, a high replay factor can cause the agent to learn from outdated experiences that do not reflect the current dynamics, leading to poor performance.

Example: Online Recommendation System: User behavior changes quickly. High ρ causes the model to over-train on old interactions, reducing adaptability to new user preferences.

v) "Planning" and DQN (With $\rho > 1$)

Planning is any computational process that uses a model (transition probabilities and rewards) of the environment to improve policies - you don't need real environment interaction, just use data from some simulations to estimate how the environment would behave.

Here, in the case that $\rho > 1$, we get only one new update, but we continue sampling ρ times (so more once) each step; meaning we are reusing older transitions to learn about current environment.

Thus DQN with $\rho > 1$ resembles planning because multiple updates are performed using stored transitions without additional environment interaction. Although no explicit model is learned, the replay buffer serves as an implicit sample-based model of the environment.

b) Distrbution of Performance across Seeds

For measuring the aggregate performance, we took the mean across the last 100 episodes in every run.

i)

The performance distributions for all replay factors appears unimodal, indicating that across seeds the algorithm typically converges to one consistent level of performance rather than distinct good/ bad modes.

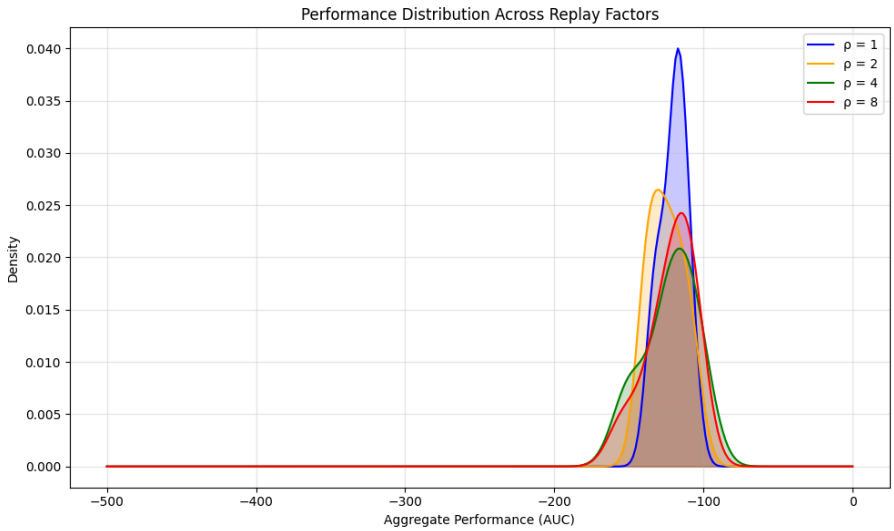

Figure 6: Density plot of final returns across seeds for different replay factors

The distributions are approximately bell-shaped, but not perfectly Gaussian. They show slight skewness, particularly for higher replay factors ($\rho = 4$ and $\rho = 8$), which have longer tails toward lower performance values. This suggests that larger replay factors occasionally produce worse runs, increasing variability. In contrast, $\rho = 1$ is more tightly concentrated, indicating more consistent performance across seeds.

Intuitively, these distributions convey that increasing ρ does not significantly change the typical performance, but it reduces reliability. Lower ρ values produce more stable and predictable learning, whereas higher ρ values occasionally lead to poorer convergence due to heavier reuse of replay data.

ii) Nuanced Inferences:

- Although the mean performances of all ρ values are similar, the distributions show that $\rho = 1$ is more tightly concentrated, meaning it is more reliable across seeds. Higher replay factors (especially $\rho = 4, 8$) have wider spreads, indicating a higher chance of poorer runs even if the average looks similar. **This is not obvious from the mean learning curves from a part alone.**
- The peaks of all distributions lie close together, showing that increasing ρ does not shift the distribution significantly to better performance. Instead, it mainly increases variance. This suggests additional replay updates do not improve the final solution quality.
- The distribution for $\rho = 2$ is slightly left-skewed compared to others. This suggests that a small increase in replay updates introduces instability without providing sufficient averaging benefits: with $\rho = 1$ updates closely track fresh data and remain stable, while larger replay factors ($\rho = 4$, 8) perform many updates that average out noise; however, $\rho = 2$ lies in between—introducing bias from replayed data but not enough updates to stabilize learning.

iii) When Visualisation is Better:

1. Safety-critical robotics (e.g. autonomous cars):
   Two algorithms may have similar mean returns, but distributions can show that one occasionally fails catastrophically. A learning curve would hide this, while the distribution reveals rare but dangerous failures, which is crucial for deployment.

2. Sparse-reward environments (e.g., maze navigation):
   Some runs may discover the goal early while others never do. The mean curve averages these together and looks smooth, but the distribution becomes multimodal (success vs failure), revealing that the algorithm is unreliable.

c) Variability in performance

i) Plots

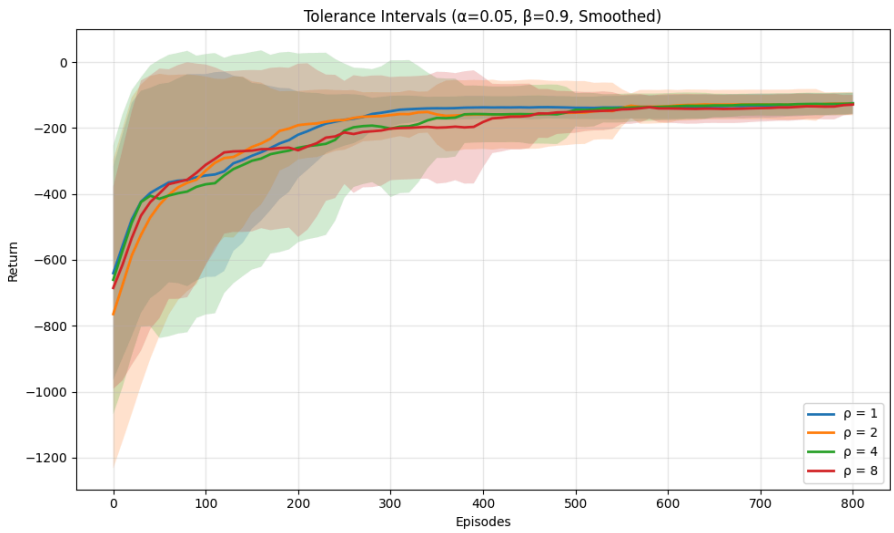

Figure 7: Tolerance Interval shadded plot of final returns across seeds for different replay factors

ii) Comparison from the plot

From the tolerance interval plot, $\rho = 1$ and $\rho = 2$ have relatively tighter bands, especially after convergence, indicating more consistent performance across runs. In contrast, $\rho = 4$ shows the widest tolerance interval during learning, suggesting high variability, while $\rho = 8$ also remains wider as well. After convergence, all intervals shrink but higher ρ values still show slightly larger spread.

This provides different information from part (a). While the mean curves suggested similar performance across ρ values, tolerance intervals reveal how much individual runs can deviate. Even when means overlap, higher ρ values show larger variability, indicating less consistent behavior.

iii) Reliability and Robustness over different ρ:

- $\rho = 1$: Most reliable, with tight tolerance intervals and consistent convergence across seeds. Hence, is the most stable choice with the best worst case performance.
- $\rho = 2$: Slightly less reliable than $\rho = 1$, with slightly wider intervals, but still generally stable.
- $\rho = 4$: Least reliable, with very wide intervals during learning, indicating high variability and potential for poor runs.
- $\rho = 8$: Also less reliable than lower ρ values, with wider intervals, though not as extreme

Thus smaller ρ values provide more consistent and robust learning, while larger ρ values introduce instability and increase the risk of poor performance in some runs, even if the average looks similar.

iv) Tolerance Intervals vs Confidence Intervals

CI (of mean estimator) does not reflect the variability of individual runs; they only indicate how precisely we have estimated the average performance.

On the other hand, tolerance intervals give a range that contains a specified proportion of individual runs with a certain confidence.It reflects spread and variability of performance, including worst-case behavior.

When Useful?:

- CIs are useful when we want to compare average performance between algorithms and understand the precision of our estimates.
- TIs are crucial when we care about the reliability and consistency of an algorithm across different runs, especially in safety-critical applications where worst-case performance matters.

TIs reveal the range of outcomes we can expect, while CIs only tell us about the average.

When Misleading?:

- CIs can be misleading if the underlying distribution is skewed or has outliers, as they may not accurately reflect the true distribution.
- TIs can be misleading if the sample size is small, as they may be too wide to provide useful information. If the data is not normally distributed, TIs may not accurately capture the variability, especially if there are outliers or heavy tails.

In the limit to infinity; the **CI** tends to the true mean, with 0 width, while the **TI** tends to the true performance distribution bounds, which is of a finite width.

d) Sensitivity Analysis

i) Batch Size

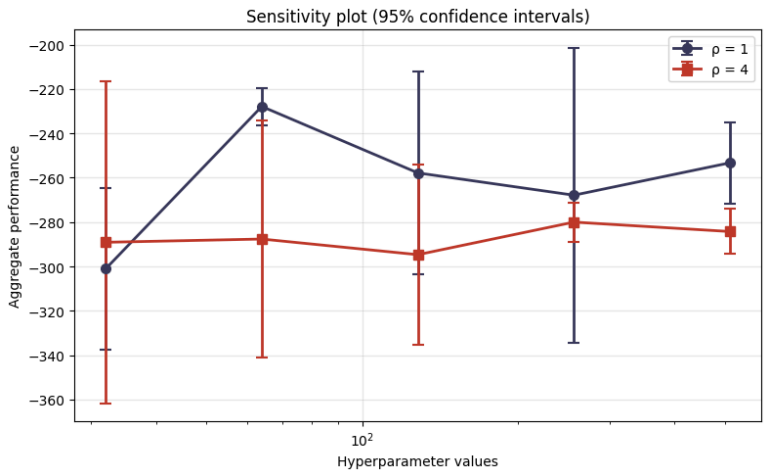

Figure 8: Batch Size Sensitivity

From the sensitivity plots, DQN with $\rho = 4$ is less sensitive to batch size than $\rho = 1$. For $\rho = 1$,batch size 64 gives the best result, while other batch sizes increase variance. This indicates that $\rho = 1$ depends strongly on choosing an appropriate batch size.

In contrast, for $\rho = 4$, performance remains relatively similar across batch sizes (64-512), with smaller differences in mean AUC and reduced variation. This suggests that increasing replay factor makes learning less sensitive to the mini-batch size, since multiple updates per step average gradient noise.

When keeping total samples per step fixed ($\rho \times$ batch-size constant), it is wiser to use a smaller ρ with a larger batch size. The results show that $\rho = 1$ with moderate/large batch sizes achieves better mean performance and lower variance than $\rho = 4$ with smaller batches. Larger batch sizes provide more stable gradient estimates, whereas increasing ρ reuses the same replay data multiple times, which does not improve performance and can introduce bias. Using smaller ρ with a larger batch size is also computationally more efficient.

ii) Target Network Refresh Rate

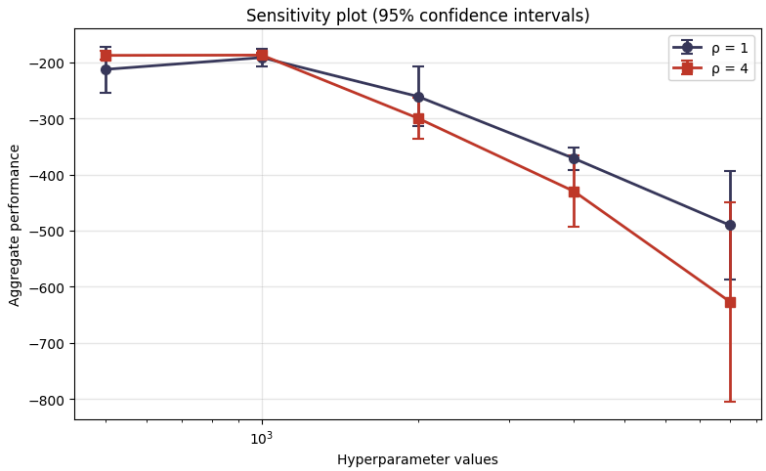

Figure 9: Target Network Refresh Rate Sensitivity

DQN with $\rho = 4$ is more sensitive to the target network refresh rate than $\rho = 1$. For both methods, very frequent updates (small refresh rate: 500–1000) lead to fast and stable learning. However, when the refresh rate becomes too large (4000–8000), performance degrades and variance increases, especially for $\rho = 4$, indicating strong instability due to stale targets.

When the refresh rate is too low (very frequent updates), learning is stable because the target closely tracks the online network. When the refresh rate is too high (infrequent updates), the target becomes outdated, leading to inaccurate TD targets and unstable learning; this effect is much stronger for $\rho = 4$ since multiple updates amplify errors from the stale target.

From the plots, $\rho > 1$ appears more stable than $\rho = 1$ for frequent updates.

When the target network is updated frequently, the TD targets change quickly and stay close to the current Q-network.

With $\rho = 4$, the agent performs multiple updates using this fresh and consistent target, so each update reinforces similar gradients. This averages noise and reduces oscillations, leading to smoother learning.

With $\rho = 1$, only one update is made per step. Since the target changes frequently, each update is based on slightly different targets, causing more fluctuation in Q-values and visible dips in performance.

iii) Real-World Scenario

In the robotics setting where collecting trajectories is expensive, increasing ρ is attractive because it allows more learning from limited real-world data. However, the hyperparameter sensitivity results show that higher ρ is also more sensitive to choices such as target update rate and batch size, and can become unstable if these are not tuned carefully. This implies that while larger ρ

improves data efficiency, it reduces robustness, making deployment riskier on real hardware where failures are costly.

In contrast, smaller ρ is more robust across hyperparameters, though it may require more environment interaction. For real-world deployment, this suggests a trade-off: larger ρ is useful when data collection is expensive, but it requires careful tuning and monitoring to avoid instability; smaller ρ is safer and more reliable when robustness is more important than minimizing interactions.

Thus, the key lesson for deployment is that higher replay factors improve sample efficiency but reduce reliability, whereas lower replay factors provide more stable and robust performance, which may be preferable in safety-critical real-world systems.

Q5 Bonus Section: PER

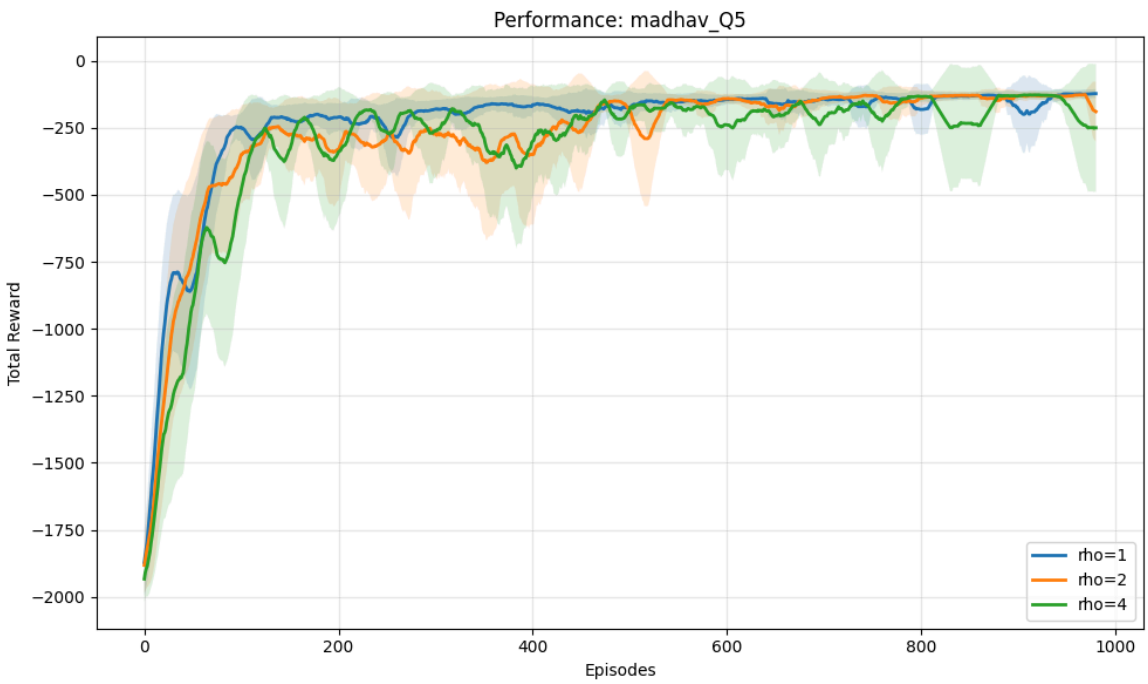

Figure 10: Reward v Episodes with different replay factors using PER

From the plot, all replay factors ($\rho = 1, 2, 4$) achieve similar final performance, and increasing ρ does not improve convergence. In fact, $\rho = 1$ learns fastest and most smoothly, while $\rho = 2$ and especially $\rho = 4$ show larger oscillations and higher variance throughout training.

With PER, increasing the replay factor leads to noticeably more oscillatory learning, especially for $\rho = 2$ and $\rho = 4$. From the plot, $\rho = 1$ improves smoothly and stabilizes, whereas $\rho = 2$ shows repeated dips after initial improvement, and $\rho = 4$ fluctuates heavily throughout training, with frequent drops and recoveries even late in learning. This suggests that the agent is repeatedly over-correcting based on a small set of high-priority transitions.

Because PER already samples transitions with large TD errors more often, using a higher ρ means those same surprising transitions are reused multiple times in succession. This can push the Q-values too aggressively in one direction, after which the TD errors change and another set of transitions becomes dominant, causing the policy to swing back. As a result, instead of converging smoothly, learning swings below the optimum, with repeated overshooting and correction cycles.